

Esame OS Internals (Cabodi)

Data: 13/01/2025

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Inverted Page Table - Vantaggi e Svantaggi)

Q1) Si considerino le affermazioni che seguono, a proposito di possibili vantaggi e svantaggi di una inverted page table (IPT), rispetto a una tabella delle pagine standard (eventualmente gerarchica) PT. Si dica di ognuno se sia vera (SI) o falsa (NO):

1. L'IPT permette di risparmiare memoria. L'IPT risparmia sempre memoria?
2. Il risparmio di memoria dell'IPT dipende dalle dimensioni della RAM, dal numero di processi e dal loro spazio di indirizzamento virtuale?
3. Con l'IPT si risparmia sempre memoria quando lo spazio di indirizzamento di un processo è maggiore della dimensione della RAM?
4. Con l'IPT si può risparmiare anche in casi in cui lo spazio di indirizzamento di ogni processo è inferiore alla dimensione della RAM?
5. L'IPT è lenta perché non garantisce accesso diretto ma occorre una ricerca. L'IPT è lenta perché occorre una ricerca in cui la chiave di ricerca è la coppia (pid, frame)?
6. L'IPT è lenta perché occorre una ricerca in cui la chiave di ricerca è la coppia (pid, pagina)?
7. Per migliorare le prestazioni si sostituisce la IPT con una tabella di HASH?
8. Per migliorare le prestazioni si aggiunge alla IPT una tabella di HASH?

(Nota: Per ognuna delle risposte alle domande precedenti, occorre fornire una breve motivazione nella domanda successiva Q2).

Domanda 2 (Esercizio Dimensionamento IPT vs PT)

Q3) Memoria Paginata:

Sia dato un processo avente spazio di indirizzamento virtuale di 48 GB, in un sistema dotato di 16GB di RAM, con architettura a 64 bit (in cui si indirizza il Byte) e gestione della memoria paginata (pagine/frame da 4KB). Si supponga che 4GB di RAM sia allocato in modo statico al kernel. Si vogliono confrontare una soluzione basata su tabella delle pagine standard (una tabella per ogni processo) e una basata su IPT.

1. Si calcolino le dimensioni della tabella delle pagine (a un solo livello) per il processo e della IPT. Si ipotizzi che il pid di un processo possa essere rappresentato su 12 bit. Si utilizzino 28 bit per gli indici di pagina e/o di frame (nella PT o nella IPT) e si tenga conto che, per allineamento, una cella di IPT o PT può solo essere di 32 o 64 bit.

2. Si dica infine, utilizzando la IPT proposta (12 bit di pid, 28 bit per un indice di pagina/frame), quale è la dimensione massima possibile per lo spazio di indirizzamento virtuale di un processo.
-

Domanda 3 (File System, Blocchi e Bitmap)

Q4) Organizzazione del Disco:

Sia dato un disco organizzato con blocchi fisici e logici di dimensione 8KB. Il disco contiene più partizioni: la partizione A, di NB blocchi, è formattata per un file system che alloca staticamente NM blocchi per i metadati (che includono directory, file control blocks e una bitmap per la gestione dello spazio libero) e ND blocchi per i dati dei file. La bitmap ha un bit per ciascuno degli ND blocchi di dati. $NM/4$ blocchi di metadati sono riservati alla bitmap. Si risponda alle seguenti domande:

1. Si calcoli il rapporto ND/NM .
 2. Supponendo che la bitmap indichi un rapporto blocchi liberi / usati del 33,33% (quindi 1 blocco libero ogni 3 usati), si calcoli (in funzione di NM) la dimensione massima per un intervallo contiguo di blocchi liberi, assumendo la configurazione più favorevole della bitmap (favorevole significa in grado di ottenere partizioni libere più grandi). Si dia la stessa risposta anche assumendo la configurazione della bitmap meno favorevole.
 3. Si supponga che un file control block (FCB) abbia dimensione 256B e $NM/4$ blocchi di metadati siano riservati agli FCB, per un massimo di 16K file. Si calcolino ND, NM e NB. Si esprima anche la dimensione della bitmap e della partizione A, espressa in Byte.
-

Domanda 4 (Linking, IPT Hashing e TLB)

Q5) Concetti Teorici:

1. Si consideri il caricamento dinamico (dynamic loading) e il link dinamico (dynamic linking). È possibile caricare dinamicamente un programma senza che sia necessario il dynamic linking? Il dynamic linking richiede che un programma sia anche caricabile dinamicamente (dynamic loading)?
2. Si spieghi brevemente perché un'Inverted Page Table necessiti di una tabella di HASH e si spieghi perché la soluzione di IPT + tabella di HASH è diversa da una soluzione con PT basata solamente su tabella di Hash.
3. Si consideri una CPU dotata di TLB: la TLB può contenere entry di più processi simultaneamente o è vincolata a contenere entry per un solo processo? Il "valid" bit presente in un entry della TLB è una semplice copia del "valid" bit presente nella Page Table? (Motivare).

2 SECTION 2: OS161

Domanda 5 (Configurazione del Kernel e Compile)

Q6) Configurazione File in OS161:

È dato un Sistema OS161. Si supponga di aver aggiunto le istruzioni seguenti a `kern/conf/conf.kern`:

```
defoption project
```

```
optfile project syscall/project.c
```

e di aver creato il file `PROJECT` in `kern/conf`, copiato dal file `DUMBVM`.

1. Si dica se le azioni descritte, più l'esecuzione (in `kern/conf`) di `./config PROJECT`, sono sufficienti affinché il file opzionale `syscall/project.c` sia compilato quando si eseguono (in `kern/compile/PROJECT`) i comandi `bmake depend` e `bmake`.
 2. Quale file, tra `project.h` e `opt-project.h` viene generato automaticamente dal comando `./config PROJECT`? Il file viene sempre generato, oppure solo se l'istruzione `options project` compare nel file `PROJECT`?
 3. Cosa contiene il file (quello generato automaticamente, citato nella domanda precedente)?
 4. Si supponga di inserire in `main.c` l'istruzione `project_init();`. Considerando che la funzione `project_init()` è implementata nel file `syscall/project.c`, come si può fare in modo che l'istruzione sia considerata e compilata solo nelle versioni del kernel in cui l'opzione `project` è abilitata?
-

Domanda 6 (Gestione Processi, Segmenti di Memoria e Syscall)

Q7) Architettura OS161 (motivare ogni risposta):

Si consideri un processo user OS161 (a ogni domanda si dia una risposta e una motivazione).

1. Quale, tra `trapframe` e `switchframe`, viene usato per gestire una system call?
2. Un `trapframe` viene allocato nello user stack o nello stack a livello kernel di un processo?
3. Perché i dispositivi di I/O sono mappati in `kseg1`? È possibile che un dispositivo di I/O sia mappato in `kseg0`?
4. Cosa succede se un processo user chiama `read(fd, buf, nb)`, dove `buf` è un indirizzo in `kseg0`? Cosa dovrebbe fare la system call `SYS_read`, al fine di gestire correttamente questo caso?

Soluzioni

Soluzione Domanda 1 (IPT)

- 1) **NO.** L'IPT ha un entry per ogni frame di memoria RAM fisica. Se il sistema ha pochissimi processi con spazi virtuali molto piccoli, una tabella delle pagine standard (specialmente se gerarchica/sparsa) occuperebbe molto meno spazio dell'intero array IPT.
- 2) **NO.** Il risparmio di memoria dipende dalla differenza tra l'occupazione della PT standard (che scala con il numero di processi e lo spazio virtuale utilizzato) e l'IPT (che scala solo con la dimensione della RAM). La domanda richiede la dipendenza da tutte le variabili, ma la risposta corretta è **VERO** se si intende che il calcolo del "risparmio" implica considerare tali dimensioni. (Nota: nella discussione precedente abbiamo corretto la logica: il risparmio effettivo dipende da quante tabelle standard vengono evitate).
- 3) **NO.** Le tabelle standard sono spesso sparse (es. paginazione multilivello). Anche se lo spazio virtuale è enorme, se solo poche pagine sono attive, la tabella standard rimane piccola, mentre l'IPT resta fissa e grande quanto la RAM.
- 4) **SI.** Se si eseguono molti processi piccoli, le tabelle standard (una per processo) si sommano, occupando molto più spazio di una singola IPT globale.
- 5) **NO.** La chiave di ricerca è (pid, pagina).
- 6) **SI.** Per identificare univocamente un frame, dobbiamo sapere a quale processo appartiene quella pagina (pid) e quale pagina sia (pagina).
- 7) **NO.** L'IPT non viene sostituita, ma potenziata.
- 8) **SI.** Una tabella di HASH riduce la complessità di ricerca da $O(N)$ a $O(1)$ mediamente, senza eliminare l'IPT che funge da memoria di riferimento.

Soluzione Domanda 2 (Dimensionamento IPT vs PT)

A) Calcolo delle dimensioni delle tabelle

Per dimensionare le tabelle, dobbiamo considerare come ciascuna struttura viene indicizzata:

1. **Tabella delle Pagine Standard (PT):** Questa struttura scala in base allo spazio di indirizzamento virtuale del processo.

- **Numero di pagine:** $\frac{48\text{G}}{4\text{K}} = \frac{48 \cdot 2^{30}}{2^{12}} = 48 \cdot 2^{18} = 3 \cdot 2^4 \cdot 2^{18} = 3 \cdot 2^{22}$ pagine.
- **Dimensione entry:** Il problema specifica 28 bit per l'indice di frame. Per l'allineamento richiesto (32 o 64 bit), dobbiamo utilizzare una cella da **32 bit** (4 Byte).
- **Calcolo finale:** $(3 \cdot 2^{22}) \text{ pagine} \cdot 4 \text{ Byte} = 12 \cdot 2^{22} \text{ Byte} = 3 \cdot 2^{24} \text{ Byte} = 48\text{M Byte}$.

2. **Inverted Page Table (IPT):** Questa struttura scala in base alla RAM fisica disponibile nel sistema.

- **Numero di frame:** $\frac{16\text{G}}{4\text{K}} = \frac{16 \cdot 2^{30}}{2^{12}} = 16 \cdot 2^{18} = 2^4 \cdot 2^{18} = 2^{22}$ frame.
- **Dimensione entry:** Ogni entry deve contenere sia il PID (12 bit) che il numero di pagina logica (28 bit). Totale = $12 + 28 = 40$ bit. Per l'allineamento, dobbiamo passare alla cella superiore da **64 bit** (8 Byte).
- **Calcolo finale:** $2^{22} \text{ frame} \cdot 8 \text{ Byte} = 2^{22} \cdot 2^3 \text{ Byte} = 2^{25} \text{ Byte} = 32\text{M Byte}$.

B) Dimensione massima dello spazio di indirizzamento virtuale

La dimensione massima dello spazio virtuale in un sistema IPT è vincolata dal numero di bit riservati all'indice di pagina nella tabella stessa.

- **Capacità di indirizzamento:** Il sistema utilizza 28 bit per l'indice di pagina, il che significa che ogni processo può indirizzare al massimo 2^{28} pagine logiche.
- **Calcolo spazio totale:**

$$\text{Spazio Max} = (\text{Numero di pagine}) \cdot (\text{Dimensione pagina})$$

$$\text{Spazio Max} = 2^{28} \cdot 4\text{K} = 2^{28} \cdot 2^{12} = 2^{40} \text{ Byte}$$

Dato che 2^{40} Byte corrispondono a 1T Byte, la dimensione massima possibile per lo spazio di indirizzamento virtuale di un processo è **1 Terabyte**.

Soluzione Domanda 3 (Organizzazione del Disco)

A) Calcolo del rapporto N_D/N_M

La dimensione della bitmap può essere espressa in due modi equivalenti:

1. **In Bit:** Poiché ogni blocco dati ha 1 bit nella bitmap, la dimensione è N_D bit, ovvero $N_D/8$ Byte.
2. **In Blocchi di Metadati:** La bitmap occupa $N_M/4$ blocchi. Poiché ogni blocco è di 8K Byte, la dimensione è $(N_M/4) \cdot 8K = 2K \cdot N_M$ Byte.

Uguagliando le due espressioni:

$$\frac{N_D}{8} = 2K \cdot N_M \implies N_D = 16K \cdot N_M \implies \frac{N_D}{N_M} = 16K$$

B) Analisi dell'intervallo contiguo

- **Configurazione Migliore:** Assumendo che tutti i blocchi liberi siano contigui. Il rapporto blocchi liberi/usati è 33,33%, ovvero per ogni 3 usati c'è 1 libero. Questo significa che i blocchi liberi costituiscono il 25% del totale N_D . Dimensione = $(N_D/4) \cdot 8K = (16K \cdot N_M/4) \cdot 8K = (4K \cdot N_M) \cdot 8K = 32M \cdot N_M$.
- **Configurazione Peggiora:** Ogni blocco libero è isolato tra blocchi occupati. La dimensione massima è pari a un singolo blocco: 8K Byte.

C) Calcoli Numerici

1. **Determinazione di N_M :** Lo spazio riservato agli FCB è $N_M/4$ blocchi. Sapendo che servono 16K FCB da 256 Byte ciascuno:

$$\text{Spazio richiesto} = 16K \cdot 256B = 4M \text{ Byte}$$

$$\text{Spazio riservato} = \frac{N_M}{4} \cdot 8K = N_M \cdot 2K$$

$$N_M \cdot 2K = 4M \implies N_M = \frac{4M}{2K} = 2048$$

2. **Blocchi Dati (N_D) e Totali (N_B):**

$$N_D = 16K \cdot N_M = 16K \cdot 2048 = 32M \text{ (ovvero } 33.554.432 \text{ blocchi)}$$

$$N_B = N_D + N_M = 33.554.432 + 2048 = 33.556.480$$

3. **Dimensioni Finali:**

- **Bitmap:** $N_M/4$ blocchi = 512 blocchi $\cdot 8K = 4M$ Byte.
- **Partizione A:** $N_B \cdot 8K = 33.556.480 \cdot 8K = 268.451.840K \text{ Byte} \approx 268,45GB$.

+

Soluzione Domanda 4 (Linking, IPT Hashing e TLB)

A) Dynamic Loading vs. Dynamic Linking

- **Dynamic Loading senza Dynamic Linking: SÌ.** È possibile caricare dinamicamente un modulo di codice (ad esempio, un driver o una specifica funzione) all'interno dello spazio di indirizzamento di un processo durante l'esecuzione, utilizzando chiamate di sistema come `dlopen()` in Unix. In questo caso, il codice caricato è "self-contained" o le sue dipendenze sono già risolte, pertanto non è richiesto il supporto del dynamic linker.
- **Dynamic Linking con Dynamic Loading: NO.** Il *dynamic linking* (risoluzione dei simboli a runtime tramite un loader) è una funzionalità fornita dal sistema operativo che può operare su programmi linkati staticamente o dinamicamente. Non è necessario che il caricamento del programma sia "dinamico" (ovvero che avvenga on-demand durante l'esecuzione della logica applicativa); il linking dinamico avviene tipicamente all'avvio del processo, indipendentemente dal fatto che i singoli moduli siano stati caricati staticamente su disco o dinamicamente in memoria.

B) IPT e Tabella di Hash

L'Inverted Page Table (IPT) mappa ogni frame di memoria fisica a una virtual page. Poiché la tabella è indicizzata per frame (fisico), una ricerca basata su un indirizzo virtuale richiederebbe una scansione lineare dell'intera IPT, rendendola di fatto inutilizzabile ($O(N)$).

- **Perché la tabella di HASH:** Si introduce una tabella di Hash per trasformare la ricerca da lineare a costante ($O(1)$), utilizzando il tuple (pid, vpn) come chiave per puntare direttamente all'entry corretta nell'IPT.
- **Differenza IPT+Hash vs. Hash Page Table:**
 - **Hashed IPT:** La struttura dati primaria rimane l'IPT (array fisico). La tabella di hash è solo un **indice** ausiliario che accelera il lookup verso l'array IPT. La sorgente di verità è l'IPT.
 - **Hash Page Table:** Non esiste un array IPT sottostante. La tabella di hash è la tabella delle pagine. Ogni bucket contiene una lista di mapping $(pid, vpn \rightarrow pfn)$. La memoria è occupata solo per le pagine effettivamente mappate, rendendola ideale per spazi virtuali vasti e sparsi.

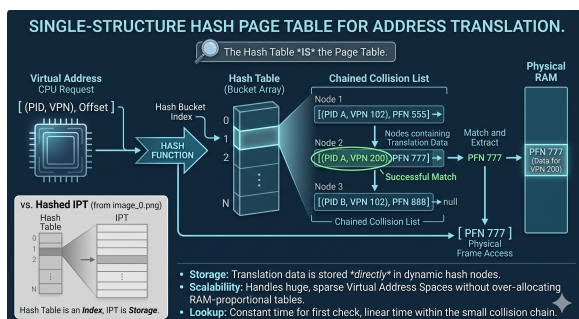


Figura 1: Hash Page Table (Storage-based)

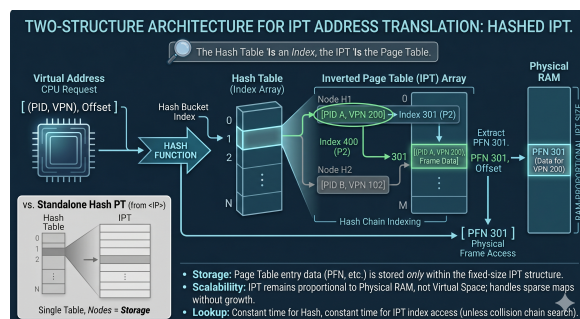


Figura 2: Hashed IPT (Index-based)

C) TLB e Context Switch

- **Processi simultanei:** In una TLB standard (senza tag PID/ASID), la TLB è vincolata a un **solo processo**. Se la TLB contenesse entry di processi diversi, non avremmo modo di distinguere se il match $VPN \rightarrow PFN$ appartenga al processo corrente o a quello precedente, causando corruzione di memoria. Per questo motivo, su switch, la TLB deve essere svuotata (*flush*).
- **Valid bit:** Il *valid bit* nella TLB **non** è una semplice copia di quello nella Page Table. Mentre il bit nella Page Table indica se una pagina è effettivamente in RAM o su disco, il *valid bit* nella TLB indica se l'entry cache è "fresca" e valida per il processo corrente. Durante un context switch, il sistema operativo invalida tutti i bit di validità della TLB (impostandoli a 0). Se durante un accesso si trova un match (tag match), ma il bit di validità è 0, l'hardware ignora il valore: questo garantisce che le traduzioni "orfane" di un processo precedente vengano scartate.